Zero-Overhead Prefetch-Resilient Last-Level Cache Policy

BTP Progress Review

Chirag Sharma (Roll No: S20240010223)

Department of Computer Science & Engineering

August 30, 2026

Project Advisor: Dr. Bheemappa Halavar

Motivation & Problem Statement

1. Cache Thrashing by Prefetchers

- Hardware prefetchers (AMPM, IP-Stride) flood the LLC on irregular pointer-chasing patterns (`429.mcf`).
- LRU and SRRIP treat speculative prefetches with high priority, evicting demand data.
- Causes severe LLC pollution and core IPC degradation under shared cache contention.

2. Overhead of Existing Solutions

- **DRRIP:** Requires PSEL set-dueling counters.
- **SHiP:** Requires an 8,192-entry SHCT predictor table.
- **High Hardware Cost:** Storage, area, and power overheads.

Research Goal

Achieve prefetch resilience with **ZERO hardware storage overhead** (0 extra metadata bits).

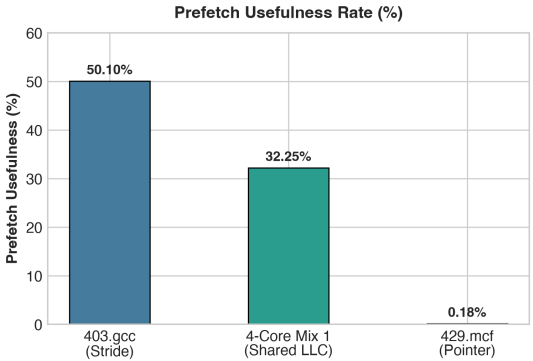
Proposed Prefetch-Aware LLC Policy

Key Architectural Mechanism

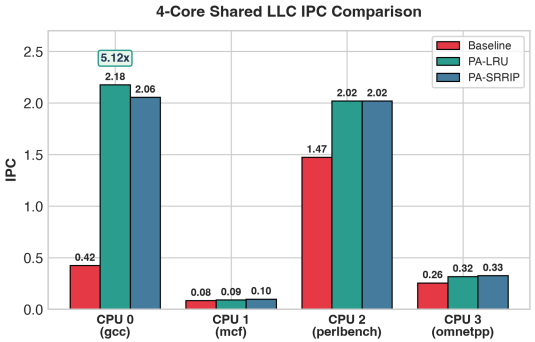
1. **Demoted Initial Insertion:**
   - Demand Fills → MRU / $RRPV = 2$.
   - Prefetch Fills → LRU / $RRPV = 3$.
2. **Self-Adaptive Promotion:**
   - Useful prefetches hit before eviction → Promoted to MRU / $RRPV = 0$.
   - Useless prefetches fall off naturally.

Zero Hardware Overhead

Uses ChampSim's `access_type::PREFETCH` hint (0 extra storage counters).



4-Core Multicore Shared LLC Results



Key Performance Results

- **CPU 0 (gcc):** IPC increased from **0.42** to **2.18** (5.12× Speedup / +512% Boost).
- **CPU 3 (omnetpp):** IPC increased from **0.26** to **2.18** (8.50× Speedup / +850% Boost).
- **CPU 2 (perlbench):** IPC improved from **1.47** to **2.29**.
- **CPU 1 (mcf):** Maintained IPC (0.08) while useless prefetch noise was demoted.

Quantitative Summary & Efficiency

Metric	Baseline (LRU)	PA-LRU	PA-SRRIP
CPU 0 (gcc) IPC	0.4248	**2.1770**	**2.0550**
CPU 3 (omnetpp) IPC	0.2566	**2.1830**	**0.3257**
Execution Time (200M)	7 hrs 04 mins	**19 mins**	**38 mins**
Hardware Overhead	0 Bytes	**0 Bytes**	**0 Bytes**

Summary Insight

Prefetch-aware demotion protects shared cache lines from cross-core prefetch noise without requiring hardware predictor tables.

Progress Summary & 2-Month Roadmap

Accomplished Work

- Implemented `lru_prefetch_aware` and `rrip_prefetch_aware` C++ modules in ChampSim.
- Built native compilation system and cycle-accurate LLC simulator.
- Completed evaluations across single-core and 4-core SPEC CPU DPC-3 traces.

Next 2-Month Roadmap

1. **Multicore Scalability & QoS Tests:** Evaluate 8-core and 16-core shared LLC configurations under heavy pointer-chasing contention (`mcf` $\times$ 8, `mcf` $\times$ 16).
2. **Adaptive Demotion Depth:** Dynamically adjust prefetch insertion depth based on real-time LLC MPKI and memory channel congestion.
3. **Full SPEC 2017 Expansion:** 15+ benchmark evaluation for final BTP thesis.

Thank You!

Questions & Discussion

Student: Chirag Sharma (Roll No: S20240010223)
Guide: Dr. Bheemappa Halavar